House of Healing

Solarpunk Apothecary

Specification — Revision 3 (Assignment 3)

GAME2401 | Spring 2026 — Michael Hardwicke

This document is maintained in lockstep with the build. Section 7 logs each revision as a decision, with the alternative that was considered and the trade-off accepted. The Assignment 1 version of this document is preserved in the repository history.

1. Game Overview

House of Healing is a single-player 3D game set inside a Solarpunk Apothecary. The player character physically operates the apothecary: collecting raw ingredients, processing them at equipment stations, combining them at a chemistry workbench, and carrying the results to a delivery shelf that stocks the building's community inventory. A day is complete when the delivery quota is met.

The game is structured around one core mechanic and its supporting loops:

- Core: Chemistry (ingredient combination and outcome evaluation)
- Supporting: Processing, Delivery, Inventory, Cleaning

Supporting loops are chores that feed and drain the chemistry system. All loops are now physical, in-world interactions rather than UI-button abstractions (Revision Log, R1).

2. Mechanics Specification

2.1 Ingredients

Six ingredient categories, each with a raw and a processed state. No ingredient can produce a successful chemistry outcome while raw. Processing happens physically at the ProcessingStation: dropped raw ingredients queue in its trigger zone, the correct strategy runs per category, and processed ingredients tint dark (placeholder visual).

Category	Type	Raw Form	Processed Form	Processing Method
Fruit	Organic	Raw fruit	Prepared fruit	Washing / cooking
Herbs	Organic	Raw herbs	Prepared herbs	Washing / drying
Oils	Organic	Raw oil	Prepared oil	Pressing / filtering
Saltpeter	Inorganic	Saltpeter	Gunpowder	Refinement
Crystals	Inorganic	Raw crystals	Powder	Grinding / filtering
Water	Solvent	Untreated water	Filtered water	Boiling / filtering

2.2 Chemistry System

The player stages 2–3 ingredients on the ChemistryWorkbench by physically placing them in its zone (picking one back up unstages it), then interacts (E) to combine. Combining is done BY the player, never TO the player: nothing evaluates until the interact. Evaluation produces exactly one of three outcome states:

- Success: the combination is a known remedy; a result item spawns beside the bench
- Neutral: a harmless known combination; the result item still spawns, and a mess is created
- Fail: a botched or dangerous mix; a mess is created (known Fail recipes still spawn their item)

Two rules are enforced by ChemistrySystem.Evaluate, in this order: (1) any raw ingredient fails the mix before any rule lookup — the spec's central rule; (2) processed ingredients matching no authored rule resolve as Fail ("Unknown Mixture"). Outcome type controls mess creation; whether a physical item spawns is a property of the authored rule — a salad is still a salad even when it isn't a remedy.

2.3 Known Combinations

Three combinations are authored as ScriptableObject assets and verified by the chemistry gym. The remaining space (15 two-ingredient pairs, 20 triples) resolves as Unknown Mixture until authored — planned Assignment 2/3 content work.

Ingredients	Outcome	Result	Mess?
Water + Powders	Success	Saline	No
Herbs + Oils	Neutral	Salad	Yes
Fruits + Gunpowder	Fail	LaxativeBob	Yes
Any raw ingredient	Fail	Ruined Mixture	Yes
Unknown processed mix	Fail	Unknown Mixture	Yes

2.4 Delivery and Win Condition

Crafted items only count when delivered. The player carries a result item to the DeliveryShelf and releases it in the shelf's zone; the shelf consumes the item, raises OnItemDelivered, and the building inventory and quota respond. A held item is ignored by the shelf — delivery is the deliberate act of setting the item down, consistent with the by-the-player rule in 2.2. The day completes when the quota (default 3) of delivered items is reached. Every delivered item counts toward quota regardless of outcome type — the building takes what it is given (design call, Revision Log R6).

2.5 Supporting Loops

- Processing: physical trigger-queue at the ProcessingStation; Strategy pattern selects behaviour per category.
- Inventory: the building inventory is the record of what has been DELIVERED, not what has been crafted; a remedy on the floor beside the bench is not in stock. Ingredient storage remains available via interaction.
- Cleaning: Neutral/Fail evaluations create a mess at the bench; interacting with the bench while it has a mess clears it before any new combination.

3. Architecture and SOLID Design

3.1 System Map

Five active systems, plus a player interaction layer. Systems live ON the equipment that owns them: the workbench prefab owns ChemistrySystem and CleaningSystem, the station prefab owns ProcessingSystem (auto-wired via GetComponent in Awake). Building-level systems (Inventory, Quota) live at scene scope. Equipment is self-contained: dropping a prefab into any scene brings its systems with it — which is what makes per-system test scenes cheap (Section 6).

System	Responsibility	Owns	Status
ProcessingSystem	Player-triggered processing across organic, inorganic, solvent categories	Strategies, state transitions	Active
ChemistrySystem	Evaluates staged combinations, spawns authored result items, raises OnCombinationResolved	Combination rules, evaluation logic, item spawning	Active
InventorySystem	Records delivered items (building inventory) and stored ingredients; raises OnInventoryChanged	Building inventory, ingredient storage	Active
CleaningSystem	Creates mess on Neutral/Fail outcomes, clears on player interaction	Mess state	Active
QuotaSystem	Counts deliveries toward the day quota; raises OnQuotaChanged / OnQuotaReached once	Quota progress	Active
Player layer	Third-person controller, hold-to-carry pickup, E-interaction (IInteractable)	Input, carry state	Active

SOLID reasoning from Revision 1 (single responsibility per system; open/closed via ScriptableObject data; Liskov via IIngredient; segregated IIngredient/IProcessable; dependency inversion through events) continues to hold and is not restated here. One addition: the player layer talks to equipment exclusively through IInteractable, so equipment never knows what a player is — the same inversion applied to input.

4. Design Patterns

4.1 Strategy: Processing (unchanged from Revision 1)

OrganicProcessingStrategy (Fruit, Herbs, Oils), InorganicProcessingStrategy (Saltpeter, Crystals), SolventProcessingStrategy (Water) behind IProcessingStrategy. Rationale and trade-off as in Revision 1.

4.2 ScriptableObject as Data Container (revised)

IngredientData and CombinationRuleData as before. Revision: CombinationLookup (the planned dictionary over rules) is CUT. Evaluate scans the rule array linearly. With a handful of authored rules, O(n) lookup against n≈3–35 buys simplicity and one less dead class; the O(1) dictionary is the documented alternative if the full 35-combination space ships and profiling says the scan matters. Trade-off stated as a commitment: we accept the scan until measurement, not intuition, argues otherwise.

4.3 Observer: Two Event Chains (revised)

Revision 1 specified one chain: chemistry raising OnCombinationResolved into the inventory. The build revealed that crafting and earning are different moments — combining creates an item in the world, but stock and quota should reflect what reaches the shelf. The single chain is now two:

- OnCombinationResolved (ChemistrySystem): consumed by CleaningSystem (mess on Neutral/Fail) and the chemistry gym. Crafting consequences.
- OnItemDelivered (DeliveryShelf): consumed by InventorySystem and QuotaSystem. Earning consequences.

Rationale: neither publisher knows its subscribers; UI, audio, or VFX can attach to either moment without modification. The alternative — keeping inventory on the combine event — was rejected because it made the physical result item decorative and the delivery loop meaningless. Trade-off: two events to trace instead of one; mitigated by per-system gyms that prove each chain fires.

4.4 MVC: Displays (as planned in Revision 1)

InventoryDisplay and QuotaDisplay are views subscribing to model events (OnInventoryChanged, OnQuotaChanged/OnQuotaReached); models never reference UI. No polling.

5. Module Boundaries and Data Flow

5.1 Interfaces

Interface	Methods	Implemented By
IIngredient	IngredientData GetData(), bool IsProcessed()	Ingredient MonoBehaviour
IProcessable	void Process()	Ingredient MonoBehaviour
IProcessingStrategy	void Execute(IProcessable target)	Organic, Inorganic, Solvent strategies
IInteractable	string GetInteractionPrompt(), void Interact(GameObject)	ChemistryWorkbench, IngredientPickup

5.2 Events

Event	Raised By	Payload	Consumed By
OnCombinationResolved	ChemistrySystem	OutcomeResult (type, result name)	CleaningSystem, ChemistryGymRunner
OnItemDelivered	DeliveryShelf	string (item name)	InventorySystem, QuotaSystem

OnIngredientProcessed	ProcessingSystem	IIngredient reference	UI feedback (future)
OnInventoryChanged	InventorySystem	—	InventoryDisplay
OnQuotaChanged / OnQuotaReached	QuotaSystem	(crafted, target) / —	QuotaDisplay

5.3 Data Flow Summary

The player collects a raw ingredient (hold-to-carry) and drops it at the ProcessingStation; the station queues it, ProcessingSystem selects the category strategy and flips the state. The player places 2–3 processed ingredients on the ChemistryWorkbench and interacts; ChemistrySystem.Evaluate applies the raw-ingredient block first, then rule lookup, raises OnCombinationResolved (CleaningSystem creates a mess on Neutral/Fail), and spawns the authored result item stamped with the rule's result name — the rule data is the single source of truth for naming. The player carries the item to the DeliveryShelf and releases it; the shelf consumes it and raises OnItemDelivered; InventorySystem records it (OnInventoryChanged updates the display) and QuotaSystem counts it (OnQuotaChanged updates the display; OnQuotaReached ends the day at the target).

5.4 Project Structure

Folder	**Contents**
Code/Core/	Interfaces (IIngredient, IProcessable, IProcessingStrategy, IInteractable, IState), OutcomeResult, enums, IngredientData, CombinationRuleData
Code/Gameplay/Systems/	ChemistrySystem, ProcessingSystem, InventorySystem, CleaningSystem, QuotaSystem
Code/Gameplay/Chores/	ChemistryWorkbench, ProcessingStation, DeliveryShelf, IngredientPickup — equipment MonoBehaviours that own their systems
Code/Gameplay/Player/	Third-person controller, camera, PlayerPickup (hold-to-carry), PlayerInteraction
Code/Utilities/	IngredientValidator (CombinationLookup cut — see 4.2)
Code/Testing/	ChemistryGymRunner, IngredientSpawner — test tooling, excluded from the game scenes
Code/UI/	InventoryDisplay, QuotaDisplay
ScriptableObjects/	Ingredients/ (6 IngredientData), Recipes/ (3 CombinationRuleData), Buckets/ (CoreUtils PrefabBucket for test tooling)
Packages/CoreUtils/	Third-party utility package; AssetBuckets used by test tooling rather than duplicating a bucket implementation

6. Testing: Gyms per System

Each system is proven in a dedicated test scene ("gym") before it is trusted in the main scene. The chemistry gym runs five authored cases on Start — the three recipes, the raw-ingredient block, and the unknown-mixture fallback — and asserts both the outcome and that the event chain fired (PASS/FAIL per case plus an event-count check). The gym scene also contains the full physical loop (player, station, bench, shelf) for hands-on verification.

Tooling shaped by observed friction: hand-placing ingredients for every test run was slow, so per-ingredient spawners (F1–F6, one key per ingredient, alphabetical) dispense raw ingredients from a shared CoreUtils PrefabBucket that watches the prefab folder and restocks itself. The bucket reuses the embedded CoreUtils package rather than duplicating existing work. The gym runner's rerun key was retired when the F row went to the spawners; the gym runs once per play session.

7. Revision Log

Each entry is a judgement: the decision, the alternative considered, and the trade-off accepted.

R1 (2026-07-07) — This is the 3D game. The build's player character, camera, and pickup layer are in scope; Revision 1's UI-button abstraction is retired. Alternative: the text-driven management sim as specified. Rejected because the physical loop is the game being built and tested; the spec follows the build's reality. Consequence: interaction layer specified in 3.1/5.1.

R2 (2026-07-07) — One chemistry design: the staged workbench. The auto-pairing trigger bench (ChemistrySystem.OnTriggerEnter) is deleted. Alternative: keep auto-combining. Rejected: combining should be done by the player, not to the player, and the staged design is what Evaluate describes.

R3 (2026-07-07) — CombinationLookup cut. Linear rule scan in Evaluate. Alternative: O(1) dictionary lookup. Rejected at current scale (3 recipes); revisit only if the 35-combination space ships and profiling demands it.

R4 (2026-07-08) — Systems live on equipment. Bench owns Chemistry+Cleaning, station owns Processing, auto-wired in Awake; Inventory and Quota stay building-level. Alternative: scene-level singletons. Rejected: self-contained equipment makes gyms drag-and-drop and keeps ownership visible in the hierarchy.

R5 (2026-07-15) — Quota counts deliveries, not combines. New DeliveryShelf raises OnItemDelivered; Inventory and Quota moved off OnCombinationResolved. Alternative: count at combine time (as first wired and demonstrated). Rejected because it made the spawned item decorative; the win condition reads "delivered to the shelf". Trade-off: a second event chain to trace (4.3).

R6 (2026-07-15) — Every delivered item counts. Outcome type governs mess creation, not quota eligibility. Alternative: only Success items count. Rejected for now: the delivery act, not the recipe quality, is the player commitment; revisit if remedies need to matter individually (a design lever, one enum check away).

R7 (2026-07-15) — Carried objects do not collide. PlayerPickup disables collision detection while carrying and restores it (with zeroed velocity) on release. Found through a recurring editor crash: a solid kinematic box glued to the player fought the character controller's depenetration every frame until the solver diverged. Consequence: triggers see items only when set down — staging and delivery both read as deliberate acts, which retroactively strengthened R5's drop-to-deliver design.

R8 (2026-07-15) — Recipe outcomes keep their variety. Saline=Success, Salad=Neutral, LaxativeBob=Fail. Alternative: author all three as Success (briefly the case in data). Rejected because it silently made Neutral unreachable and hid the mess loop; with R6, variety costs nothing against quota. The drift between the gym's expectations and the data was caught in review — the gym now asserts the authored data exactly.

R9 (2026-08-21) — State machine introduced; IState implemented for the first time. StateMachine.cs added to Core/, four states added under Gameplay/States/, GameManager added as the context object and the only script that calls SceneManager. Alternative: a single GameManager with an enum and a switch, which is fewer files. Trade-off: more files, in exchange for a driver that holds no game knowledge and can be reused by writing new IState implementations rather than by editing it.

R10 (2026-08-21) — Day timer added as the lose condition. dayLengthSeconds on GameManager, counted down in PlayingState. Alternative: an order queue with per-order timers. Trade-off: one clock instead of many, because the finite ingredient stock that would make per-order pressure meaningful was not built. A per-order timer with unlimited ingredients is pressure with no decision attached.

R11 (2026-08-21) — Win check runs before the timeout check. PlayingState.Update tests the quota first and returns before touching the clock. Trade-off: filling the quota on the final frame is a win rather than a tie. Arbitrary, but it has to be decided somewhere, and deciding it in the player's favour is the version that does not read as a bug.

R12 (2026-08-21) — Restart implemented as a scene reload rather than a reset method. GameManager.Restart calls SceneManager.LoadScene on the active build index. Alternative: a ResetForNewDay method on each system. Trade-off: a reload is coarser and costs a load, but a system added later cannot be forgotten in a reset method nobody remembers to update. Every listener already unsubscribes in OnDisable, so the reload is clean.

R13 (2026-08-21) — Interaction prompt drawing changed from per-pass construction to cached. PlayerInteraction.OnGUI built a GUIStyle and an interpolated string on every pass, and OnGUI runs several times per frame. The style is now built once, and the content and measured rect are rebuilt only when the label text or the window size changes. Trade-off: none. The first version of this change cached per interaction target rather than per label, which froze the station's queued count; caching on the text corrects it.

R14 (2026-08-21) — Spawner objects removed from the game scene. The six Decor_Spawner_* objects are gone from Main. Reason: this specification already describes them as test tooling excluded from game scenes, so their presence contradicted the document. Their intended replacement, a fixed manifest of ingredients delivered at day start, was not built, so ingredient supply is currently whatever is hand-placed in the level.

R15 (2026-08-21) — Input System migration completed; every legacy Input call removed. PlayerPickup, PlayerInteraction, IngredientSpawner and the new state classes now read Mouse.current and Keyboard.current. GameManager stores its bindings as Key rather than KeyCode.

Reason: activeInputHandler is set to 1, Input System only, and in that mode the legacy Input class throws rather than returning false. PlayerPickup was entirely legacy, so hold-to-carry was throwing on every press and no ingredient could be carried or delivered. Trade-off: none, this was a defect.

R16 (2026-08-21) — Input components unsubscribe through a cached reference instead of the singleton. PlayerLocomotionInput, PlayerActionsInput and ThirdPersonInput capture the PlayerControls object in OnEnable and use that reference in OnDisable. Alternative: silencing the error log and leaving the lookup in place. Trade-off: none. Unity destroys scene objects in an unspecified order, so the input manager can be gone before its subscribers shut down; a component that holds its own reference to what it subscribed to can always unsubscribe.

8. Divergence Summary

Where the Assignment 3 build differs from this specification. Listed so the document and the build can be compared directly rather than assumed to agree.

The recipe authoring and validation tool is not built. Section 6 describes tooling shaped by observed friction; the A3 tool was specified as an editor window listing CombinationRuleData assets, creating new ones from an ingredient picker, and flagging duplicate rule sets, missing ingredient references and unset outcome enums. None of it exists. Recipes are authored directly in the Inspector and nothing validates them before runtime. The multiset match in CombinationRuleData.Matches means two rules can be logically identical while looking different in the Inspector, and that is the case the missing validator was meant to catch.

Ingredient supply is not finite. The day structure above assumes a bounded set of ingredients per day. The build has no day-start manifest; supply is whatever is hand-placed in the level, so the day timer is the only pressure in a run.

The run loop is new and is not described anywhere above section 7. Revision 2 has no Ready, Won or Lost state, no day length and no restart. Sections 1 and 2.4 should be read as describing the Playing state only.

ChemistrySystem.Evaluate still instantiates the result item. The Assignment 2 architecture review proposed splitting this so Evaluate returns an OutcomeResult and a listener spawns the physical item. That change was planned for this revision and not made, so evaluation and spawning remain in one method.

The run's own UI is OnGUI, not a Canvas. Section 4.4 describes InventoryDisplay and QuotaDisplay as views over model events, and that is still accurate for those two. The Ready, timer and end-of-run banners drawn by GameManager, and the interaction prompt drawn by PlayerInteraction, are immediate-mode and are not part of that MVC arrangement. They need no scene setup, which is what makes the loop droppable into any scene, but they are not the final presentation layer.

Interaction scripts were placed on child objects rather than prefab roots. ProcessingStation, ChemistryWorkbench and DeliveryShelf were attached to a child of each prefab, so the player's overlap search did not reach them and no station could be interacted with. Corrected during

Assignment 3 testing. PlayerInteraction now searches with GetComponentInParent so a collider on a child mesh still resolves to the script on the root.

The specification does not describe an input layer, and one exists. Input is the Input System package only (activeInputHandler 1), with a PlayerInputManager owning a generated PlayerControls object and three action maps: PlayerLocomotionMap, ThirdPersonMap and PlayerActionsMap.

9. Out of Scope

Deferred to later assignments: undoable actions (Command pattern — natural fit for crafting actions once the loop is stable), outcome VFX beyond debug text, pause, title/main menu, tutorial, settings.

Out of scope for all three assignments: audio and particle effects; save/load systems.